

07_Spark_Integridad_Distribuida_executed

March 11, 2026

1 Práctica 4: Gestión de Integridad en Arquitecturas Distribuidas

Ref. Teórica: UT3 - Punto 4 (Gestión de integridad en arquitecturas distribuidas) **Tecnología:** Apache Spark (Modo Local)

En sistemas distribuidos, factores como particiones de red, caídas de nodos o productoras desincronizadas generan riesgos de **integridad**. Como vimos en la **UT3 - 4.1**, la integridad se asegura combinando **responsabilidades técnicas** (idempotencia en Spark) y **organizativas** (contratos de datos empíricos).

En esta práctica vamos a simular fallos comunes abordados en la teoría y a resolverlos con Spark puro en la Arquitectura Medallón.

```
[1]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, \
    DoubleType, TimestampType

spark = SparkSession.builder \
    .appName("Practica_Integridad_UT3") \
    .master("local[*]") \
    .getOrCreate()

# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.")
```

Versión de PySpark 3.5.0.

1.0.1 Caso 1: Duplicados por Reintentos de Red

(Ref. Teórica UT3 - 4.3 Ejemplo 1: Duplicados en compras)

Problema: La red falla a mitad del envío. El sistema origen (App de pagos) no recibe el *ACK*

(Acknowledgement) y “por si acaso” vuelve a enviar la transacción. Como resultado, la **Capa Bronce** ingesta el mismo evento `payment_id` dos veces.

```
[2]: from datetime import datetime
# Simulamos los datos que acaban de llegar a nuestra Capa Bronce (Nota el ↵
    ↪payment_id 'P-101' duplicado)
datos_bronce = [
    ("P-100", "userA", 50.0, datetime(2026, 1, 1, 10, 0)),
    ("P-101", "userB", 20.0, datetime(2026, 1, 1, 10, 5)), # ORIGINAL
    ("P-101", "userB", 20.0, datetime(2026, 1, 1, 10, 5)), # RETRY FANTASMA
    ("P-102", "userA", 15.0, datetime(2026, 1, 1, 10, 10))
]
columnas = ["payment_id", "user_id", "amount", "timestamp"]

df_crudo = spark.createDataFrame(datos_bronce, columnas)
print(" Crudo en Capa Bronce:")
df_crudo.show()

# PREGUNTA: ¿Qué pasa si el equipo de BI (Capa Oro) suma esto directamente?
print(" Ingresos FALSOS en Capa Oro sin deduplicar:")
df_crudo.agg(sum("amount")).show()
```

Crudo en Capa Bronce:

payment_id	user_id	amount	timestamp
P-100	userA	50.0	2026-01-01 10:00:00
P-101	userB	20.0	2026-01-01 10:05:00
P-101	userB	20.0	2026-01-01 10:05:00
P-102	userA	15.0	2026-01-01 10:10:00

Ingresos FALSOS en Capa Oro sin deduplicar:

sum(amount)
105.0

Solución (Capa Plata): Idempotencia con `dropDuplicates` El Data Engineer debe garantizar que el pipeline sea idempotente (procesar el mismo archivo mil veces no cambia el resultado).

```
[3]: # Construimos nuestra tabla Plata asegurando la unicidad del identificador ↵
    ↪transaccional
df_plata_idempotente = df_crudo.dropDuplicates(["payment_id"])

print(" Datos reales (Plata):")
```

```
df_plata_idempotente.show()

print(" Ingresos REALES (Oro):")
df_plata_idempotente.agg(sum("amount")).show()
```

```
Datos reales (Plata):
+-----+-----+-----+-----+
|payment_id|user_id|amount|          timestamp|
+-----+-----+-----+-----+
|      P-100|  userA|  50.0|2026-01-01 10:00:00|
|      P-101|  userB|  20.0|2026-01-01 10:05:00|
|      P-102|  userA|  15.0|2026-01-01 10:10:00|
+-----+-----+-----+-----+
```

```
Ingresos REALES (Oro):
+-----+
|sum(amount)|
+-----+
|         85.0|
+-----+
```

1.0.2 Caso 2: Cierre Diario vs Late Events

(Ref. Teórica UT3 - 4.3 Ejemplo 2: Datos tardíos rompen el cierre diario)

Problema: Tienes un Job programado que calcula “Las ventas de ayer”. Pero un nodo de una tienda se quedó sin Wi-Fi y manda a tu servidor hoy las ventas de... ayer tras horas de latencia. Si sobrescribes todo, jodes la contabilidad histórica (el Dashboard de ayer “cambia” solo).

```
[4]: from pyspark.sql.functions import to_date

# 1. Este es nuestro histórico Oro calculado ayer (3 de Enero)
kpi_historico_oro = [
    ("2026-01-02", 5000.0), # Ayer calculamos 5K
    ("2026-01-01", 3000.0)
]
df_historico_oro = spark.createDataFrame(kpi_historico_oro, ["fecha_venta",
    ↪ "total_facturado"])
print(" Histórico Oficial en Base de Datos (Oro):")
df_historico_oro.show()

# 2. Hoy (4 de Enero), nos llega un pequeño Excel rezagado (Late Event) con
    ↪ transacciones del día 2
datos_tardios = [
    ("P-999", "userC", 100.0, datetime(2026, 1, 2, 23, 50)) # Ojo, es del día 2!
```

```
]
df_tardio = spark.createDataFrame(datos_tardios, ["payment_id", "user_id", "amount", "timestamp"]) \
    .withColumn("fecha_venta_rezagada", to_date("timestamp"))
print(" Eventos Tardíos llegados a Plata hoy:")
df_tardio.show()
```

Histórico Oficial en Base de Datos (Oro):

```
+-----+-----+
|fecha_venta|total_facturado|
+-----+-----+
| 2026-01-02|          5000.0|
| 2026-01-01|          3000.0|
+-----+-----+
```

Eventos Tardíos llegados a Plata hoy:

```
+-----+-----+-----+-----+-----+
|payment_id|user_id|amount|          timestamp|fecha_venta_rezagada|
+-----+-----+-----+-----+-----+
|      P-999|  userC| 100.0|2026-01-02 23:50:00|          2026-01-02|
+-----+-----+-----+-----+-----+
```

Solución: Reconciliación o Reproceso Controlado Defines un SLA: “Los datos son definitivos pasadas 72hs”. Si llega algo en ese margen, el pipeline debe hacer un *Merge* o reprocesar explícitamente solo esa partición de fecha en la tabla `df_historico_oro`.

1.0.3 Caso 3: Cambio de Esquema Silencioso y Corrupción

(Ref. Teórica UT3 - 4.3 Ejemplo 3: Cambio de esquema silencioso)

Problema: En origen, siempre enviaban el peso del paquete como número. Una actualización en su App sin avisarnos ahora lo envía como texto ("14 Kg"). Si usamos lecturas relajadas (como CSV con `inferSchema=True`), Spark intentará parsear “14 Kg” y al fallar matemáticamente, escribirá Null sin detener el Job. **¡Basura entra, Basura sale!**

```
[5]: # 1. Archivo que nos mandó la App engañándonos
# Simulamos que esto estaba escrito en CSV:
datos_corruptos = [
    ("Package1", "14"),
    ("Package2", "15 Kg"), # AQUÍ ESTÁ EL FALLO DE ORIGEN (String en vez de
    ↪Double)
    ("Package3", "12")
]
df_bad = spark.createDataFrame(datos_corruptos, ["product", "weight_kg"]) #
    ↪Spark lo traga como String
```

```
print(" Lo que pasa si el catálogo de metadatos o el contrato no tiene Schema_
↳Fijo:")
# Intentar hacer matemáticas sobre peso de repente te dará Nulls
df_bad.select(col("product"), col("weight_kg").cast("double")).show()
```

Lo que pasa si el catálogo de metadatos o el contrato no tiene Schema Fijo:

```
+-----+-----+
| product|weight_kg|
+-----+-----+
|Package1|      14.0|
|Package2|      NULL|
|Package3|      12.0|
+-----+-----+
```

Solución (Capa Plata): Contratos de Datos (Schema Structs) Tu código en Spark debe imponer un contrato estricto (**StructType**). De esta forma, si el origen cambia el esquema, Spark lanzará una alerta en la fase de testeo de calidad (o guardará el registro en cuarentena / Dead-Letter-Queue) antes de envenenar la tabla de ventas de tu Analista de Negocio.

```
[6]: # Ejemplo de un contrato estricto, definido por el Data Engineer en Metadatos
# (Es la única forma de garantizar semántica)
contrato_estriicto = StructType([
    StructField("product", StringType(), False), # Falso = NO Nulos permitidos
    StructField("weight_kg", DoubleType(), True)
])

print(" Usar .schema(contrato_estriicto) en tu comando spark.read.csv() obliga_
↳a Spark a validar la estructura.")
```

Usar `.schema(contrato_estriicto)` en tu comando `spark.read.csv()` obliga a Spark a validar la estructura.

```
[7]: spark.stop()
```